



UNIVERSIDAD AUTÓNOMA DE SINALOA



Facultad de Informática Culiacan
Licenciatura en Informática

Desarrollo Web Lado Servidor

Actividad:

API clientes con NodeJS

Docente:

MC. Jose Manuel Cazares Alderete

Alumno:

Joel Enrique Angulo Calleros

Grupo: 2-1

Culiacán, Sinaloa, 8 de Junio del 2026.

El siguiente código tiene como objetivo permitir la visualización de información a través de diferentes rutas, las cuales pueden ser consultadas utilizando la herramienta Postman.

```
API CLIENTES > JS Server.js > ...
1  const express = require('express')
2  const app = express()
3  const port = 3000
4  const fs = require('fs')
5
6  app.listen(port, () => {
7    | console.log(`Servicio Iniciado`)
8  | })
9
10 //Muestra todos los creditos activos
11
12 app.get('/creditos', (req,res) => {
13   | const contenido = fs.readFileSync('clientes.json')
14   | const data= JSON.parse(contenido)
15   | const clientes = data.filter(cliente => cliente.activo === true);
16   | res.send(clientes)
17 | })
18
19 //Muestra todod los creditos activos mayores al ingresado
20
21 app.get('/creditos/:importe', (req,res) => {
22   | const contenido = fs.readFileSync('clientes.json')
23   | const data = JSON.parse(contenido)
24   | const importe = Number(req.params.importe);
25   | const credito = data.filter(cliente => cliente.activo === true && cliente.credito > importe);
26   | res.send(credito)
27 | })
28
29 //Muestra informacion de un cliente especifico
30
31 app.get('/creditos-cliente/:id', (req,res) => {
32   | const { id } = req.params;
33   | const contenido = fs.readFileSync('clientes.json')
34   | const data = JSON.parse(contenido)
35   | const encontrado = data.find((x) => x.id == id)
36   | if (!encontrado){
37   |   | res.status(404).json({mensaje: 'No encontrado'})
38   | } else {
39   |   | res.send(encontrado)
40   | }
41 | })
42
```

La ruta **"/creditos"** permite mostrar todos los créditos que se encuentran en estado activo dentro del archivo **clientes.json**. Para lograr esto se utiliza el método **data.filter**, el cual se encarga de revisar todos los registros y seleccionar únicamente aquellos clientes que tienen créditos activos.

```
9
10 //Muestra todos los creditos activos
11
12 app.get('/creditos', (req,res) => {
13   const contenido = fs.readFileSync('clientes.json')
14   const data= JSON.parse(contenido)
15   const clientes = data.filter(cliente => cliente.activo === true);
16   res.send(clientes)
17 })
```

Por otro lado, la ruta **"/creditos/:importe"** muestra la información de los clientes que poseen un crédito activo cuyo monto sea mayor al valor que se introduce como parámetro. Por ejemplo, si se ingresa un importe de **5000**, el sistema mostrará únicamente los créditos activos que superen esa cantidad. Nuevamente se utiliza **data.filter** para analizar los datos y filtrar solo aquellos que cumplan con las condiciones establecidas.

```
//Muestra todos los creditos activos mayores al ingresado

app.get('/creditos/:importe', (req,res) => {
  const contenido = fs.readFileSync('clientes.json')
  const data = JSON.parse(contenido)
  const importe = Number(req.params.importe);
  const credito = data.filter(cliente => cliente.activo === true && cliente.credito > importe);
  res.send(credito)
})
```

En la ruta **"/creditos-cliente/:id"**, el sistema busca la información de un cliente específico utilizando su **id**. En este caso, el parámetro **id** debe ser un número. Si el cliente existe dentro del archivo, el método **data.find** se encarga de localizarlo y devolver su información correspondiente.

```
29 //Muestra informacion de un cliente especifico
30
31 app.get('/creditos-cliente/:id', (req,res) => {
32   const { id } = req.params;
33   const contenido = fs.readFileSync('clientes.json')
34   const data = JSON.parse(contenido)
35   const encontrado = data.find((x) => x.id == id)
36   if (!encontrado){
37     res.status(404).json({mensaje: 'No encontrado'})
38   } else {
39     res.send(encontrado)
40   }
41 })
42
```

Finalmente, existen algunas líneas de código cuya función es indicar que el servicio está funcionando correctamente. La primera línea se encarga de importar el archivo que contiene los datos, mientras que la segunda línea convierte esa información en un objeto de JavaScript para poder manipularla dentro del programa.

```
app.listen(port, () => {
  console.log(`Servicio Iniciado`)
})

const contenido = fs.readFileSync('clientes.json')
const data= JSON.parse(contenido)
```